



Visual Studio Code

Your Integrated Development Environment

Chapter Overview

- 01** Introduction to IDEs
- 02** Getting Started
- 03** The Editor
- 04** Integrated Terminal
- 05** Extensions
- 06** Debugging
- 07** Git Integration
- 08** Productivity & Settings
- 09** Class Exercises



Introduction to IDEs

Section 1



What is an IDE?

Definition

- An Integrated Development Environment (IDE) combines multiple development tools into a single application
- Replaces separate tools for editing, compiling, debugging, and testing

Core Features

- Source code editor with syntax highlighting
- Integrated terminal and build tools
- Debugger with breakpoints and variable inspection
- Version control integration (Git)



Why Visual Studio Code?

- Free and open source — backed by Microsoft
- One of the most popular IDE — used by 73% of developers (2024 Stack Overflow developer survey)
- Cross-platform — Windows, macOS, and Linux
- Lightweight but powerful — fast startup, rich features
- Massive extension ecosystem
- Built-in Git support, integrated terminal, and debugger



Getting Started

Section 2



Installing VS Code

Download

- Visit <https://code.visualstudio.com/download>
- Choose installer for your operating system

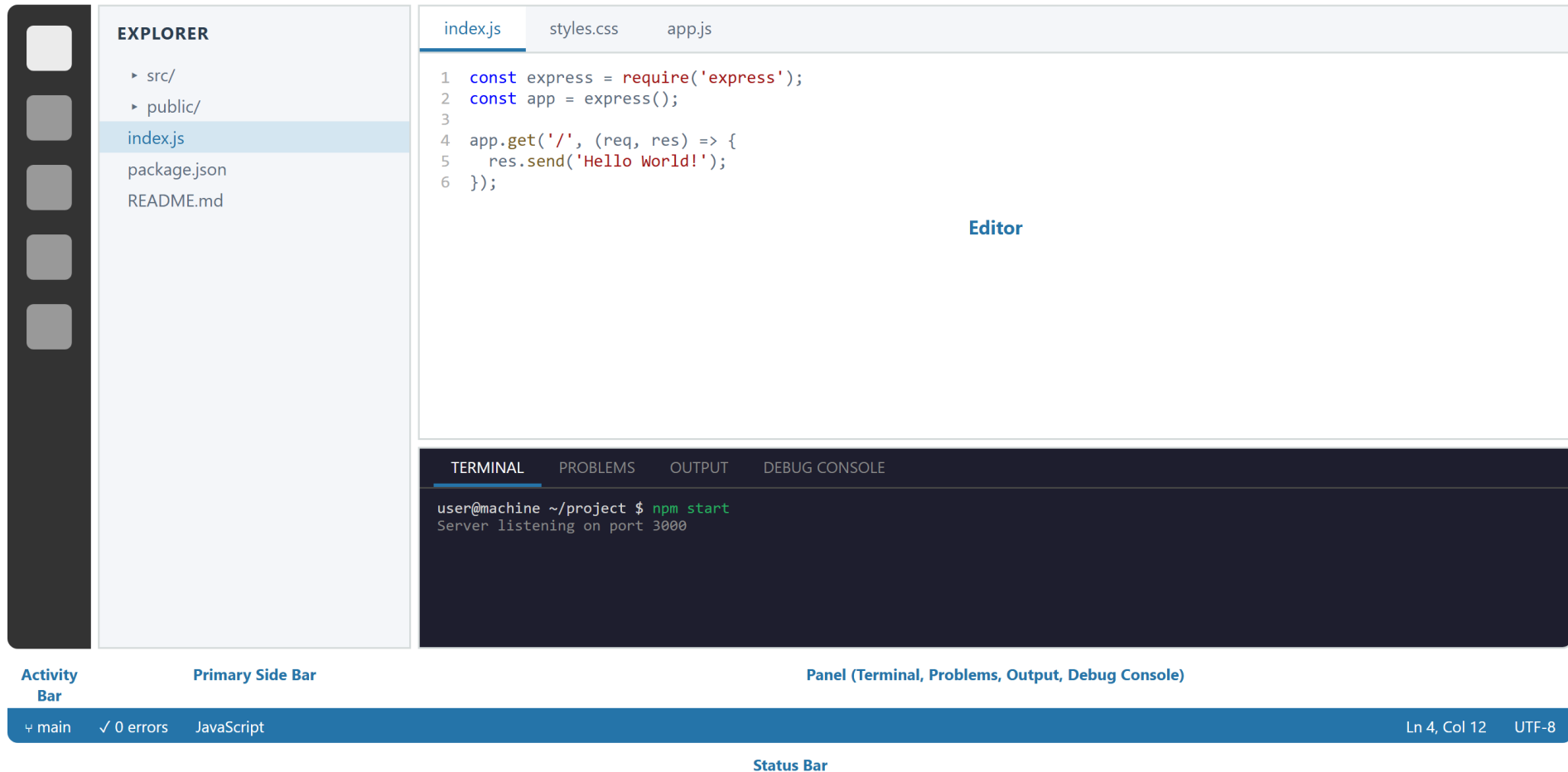
Recommended Settings During Install

- Add 'Open with Code' to file and folder context menus
- Register VS Code as default editor for supported file types
- Add to PATH for command-line access (code . to open a folder)



VS Code User Interface

VS Code User Interface



The Command Palette

Your Gateway to Everything

- Open with Ctrl+Shift+P — the most important shortcut to learn
- Access any command, setting, or feature by name
- Quick Open (Ctrl+P) — open any file by typing part of its name

Common Uses

- Change color theme: type 'color theme'
- Format document: type 'format document'
- Toggle word wrap, change language mode, open settings



Settings & Customization

Accessing Settings

- Ctrl + , opens the Settings UI (Cmd + , on macOS) — search any setting by name
- Settings can also be edited as JSON for precise control

Key Settings to Configure

- Font size and font family
- Auto-save (`files.autoSave: afterDelay`)
- Tab size and indentation (`editor.tabSize`)
- Format on save (`editor.formatOnSave: true`)



The Editor

Section 3



Editor Basics

Tabs & Split Editors

- Open multiple files in tabs — drag to rearrange
- Split editor: Ctrl+\ to work on two files side by side

Navigation Features

- Minimap — scrollable overview of your file on the right edge
- Breadcrumbs — file path and symbol navigation at the top
- Go to Definition (F12) — jump to where a function is defined
- Peek Definition (Alt+F12) — see definition inline without leaving



IntelliSense & Auto-Completion

What It Does

- Suggests completions as you type — variables, methods, keywords
- Shows function signatures, parameter info, and documentation
- Works out of the box for JavaScript, TypeScript, JSON, HTML, CSS

Using IntelliSense

- Trigger manually with Ctrl+Space
- Accept suggestion with Tab or Enter
- Install language extensions for Python, Java, C#, etc.



Multi-Cursor Editing

```
# Multi-cursor lets you edit multiple lines simultaneously

# Place cursors: Alt+Click to add cursors at specific positions

# Select next match: Ctrl+D
# Example: Select "name" then press Ctrl+D to select the next "name"

# Select all occurrences: Ctrl+Shift+L
# Edits every instance at once – great for renaming variables

# Column/box selection: Shift+Alt+Drag
# Select a rectangular region of text

# Add cursor above/below: Ctrl+Alt+Up/Down
```



Find & Replace

```
# Find in current file
Ctrl+F          Open find dialog
Ctrl+H          Open find and replace
F3 / Shift+F3   Find next / previous match

# Find across all files
Ctrl+Shift+F    Search in all project files
Ctrl+Shift+H    Replace across all files

# Pro tips:
- Toggle regex mode (.* ) for pattern matching
- Toggle case-sensitive (Aa) for exact matches
- Toggle whole word (ab) to avoid partial matches
- Use capture groups: find "(\w+)@" replace "$1_at_"
```



Integrated Terminal

Section 4



Using the Integrated Terminal

Why Use the Integrated Terminal?

- No need to switch between VS Code and an external terminal
- Automatically opens in your project's root directory
- Supports multiple terminal instances side by side

Key Shortcuts

- Ctrl+` (backtick) — toggle terminal panel
- Ctrl+Shift+` — create a new terminal instance

Terminal Profiles

- Choose PowerShell, Command Prompt, Git Bash, or WSL
- Set your preferred default in settings



Terminal in Action

```
# Run your project directly from the integrated terminal
```

```
$ npm install      # Install dependencies  
$ npm start        # Start development server
```

```
$ python app.py    # Run a Python script
```

```
$ git status       # Check Git status  
$ git add .        # Stage changes  
$ git commit -m 'msg' # Commit changes
```

```
# Split terminals for running server + watching tests  
# Click the split icon or use the terminal dropdown
```



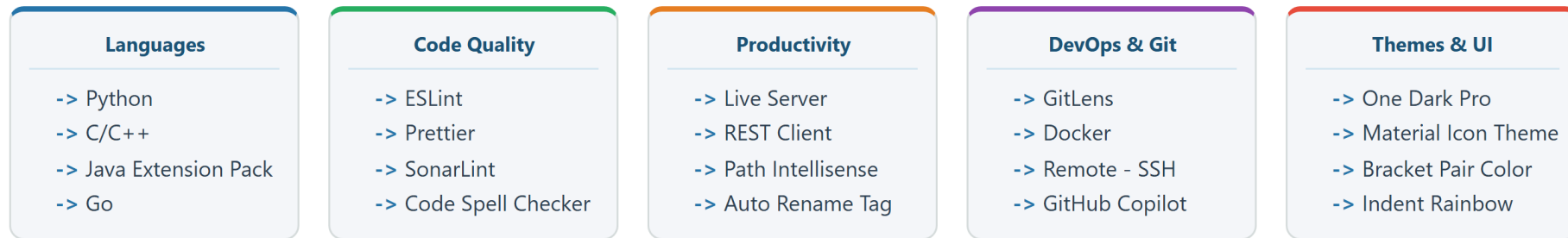
Extensions

Section 5



Extension Ecosystem

Extension Ecosystem



40,000+ extensions available on the VS Code Marketplace — install via Ctrl+Shift+X



Essential Extensions

For Web Development

- Live Server — launch a local dev server with live reload
- Prettier — automatic code formatting on save
- ESLint — JavaScript linting and error detection

For All Developers

- GitLens — enhanced Git blame, history, and annotations
- Code Spell Checker — catch typos in code and comments
- Path Intellisense — autocomplete file paths



Managing Extensions

Installing Extensions

- Open Extensions view: Ctrl+Shift+X
- Search by name, browse categories, or filter by popularity
- Click Install — extension activates immediately

Team Sharing

- Create .vscode/extensions.json to recommend extensions to the team
- VS Code prompts team members to install recommended extensions

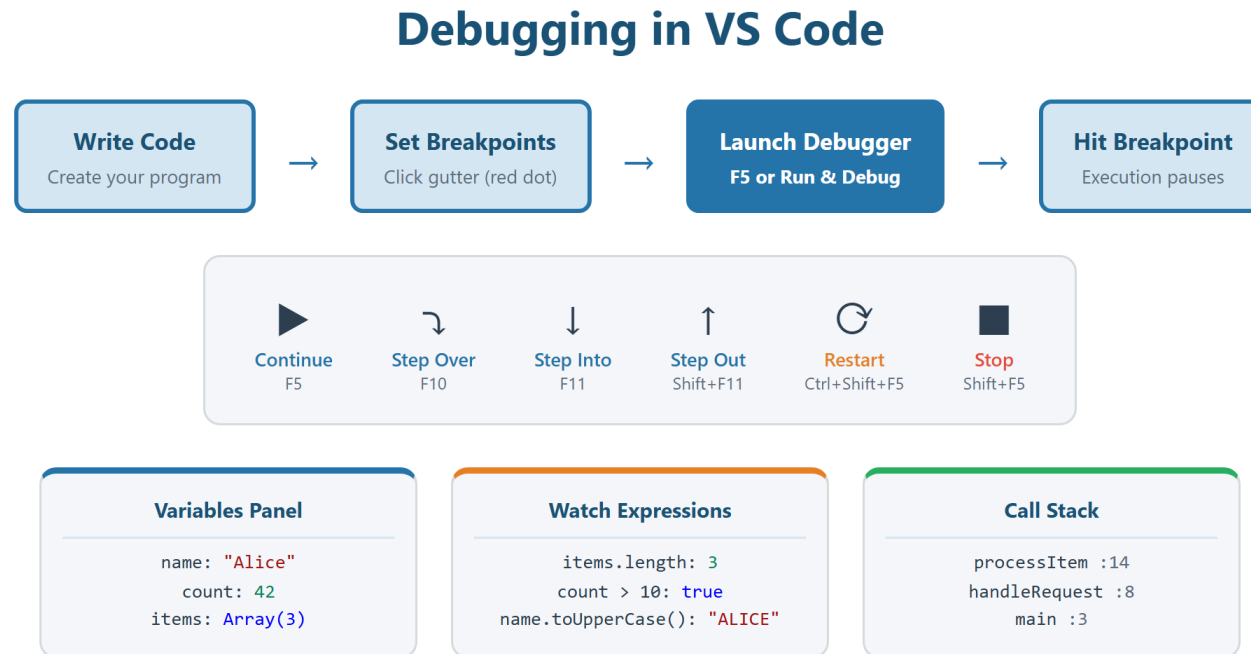


Debugging

Section 6



Debugging in VS Code



Breakpoints & Debug Controls

Setting Breakpoints

- Click the gutter (left of line numbers) to add a red dot
- Conditional breakpoints: right-click gutter → add expression
- Logpoints: log a message without stopping (right-click → Logpoint)

Debug Panels

- Variables — view and modify local/global variable values
- Watch — add custom expressions to monitor
- Call Stack — see the chain of function calls
- Debug Console — evaluate expressions while paused



Debug Configuration (launch.json)

```
// .vscode/launch.json – configures how the debugger runs
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "node",
      "request": "launch",
      "name": "Run Current File",
      "program": "${file}",
      "console": "integratedTerminal"
    },
    {
      "type": "node",
      "request": "launch",
      "name": "Run App",
      "program": "${workspaceFolder}/app.js"
    }
  ]
}
```

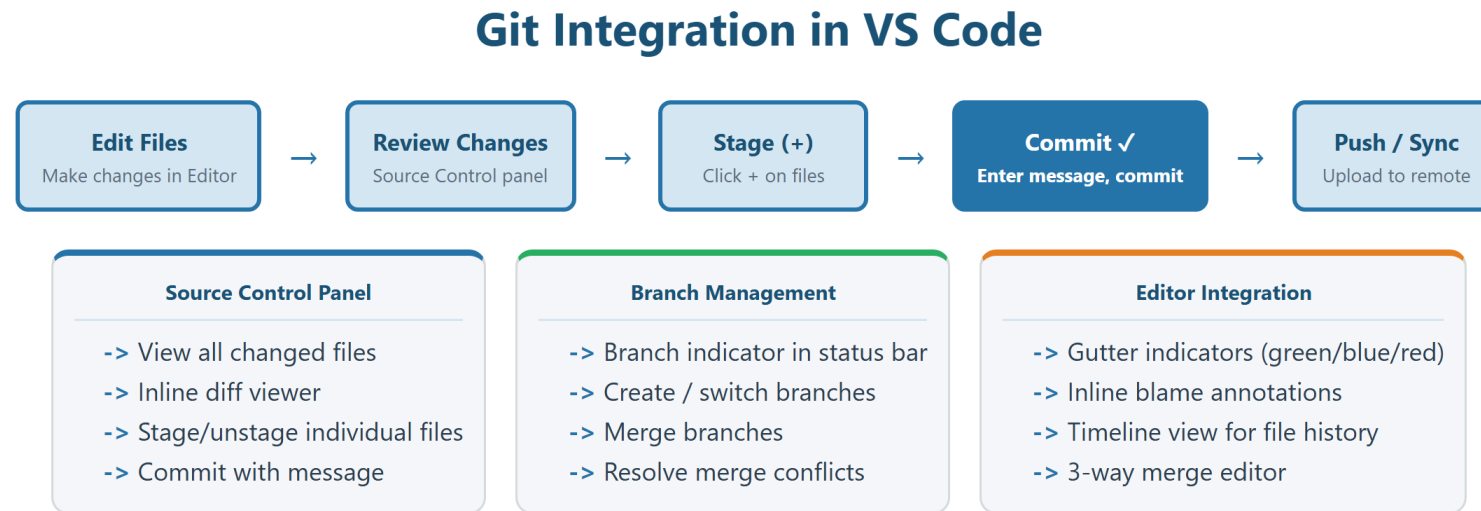


Git Integration

Section 7



Git Integration in VS Code



Access via Activity Bar icon or `Ctrl+Shift+G` — Enhanced with **GitLens** extension for advanced blame, history, and comparisons

Source Control Features

Everyday Git Workflow

- Source Control icon shows pending change count in badge
- Click a file to see inline diff — old vs. new side by side
- Stage files with + button, unstage with – button
- Type commit message and click ✓ to commit

Branch & Merge

- Current branch shown in status bar — click to switch or create
- 3-way merge editor helps resolve conflicts visually



Productivity & Settings

Section 8



Essential Keyboard Shortcuts

Essential Keyboard Shortcuts

General	
Ctrl+Shift+P	Command Palette
Ctrl+P	Quick Open file
Ctrl+,	Settings
Ctrl+`	Toggle Terminal
Ctrl+B	Toggle Side Bar

Editing	
Ctrl+D	Select next match
Alt+↑/↓	Move line up/down
Shift+Alt+↓	Copy line down
Ctrl+Shift+K	Delete line
Ctrl+/ Ctrl+`	Toggle comment

Navigation	
Ctrl+G	Go to line
Ctrl+Shift+O	Go to symbol
F12	Go to definition
Alt+←/→	Navigate back/fwd
Ctrl+Shift+F	Search all files

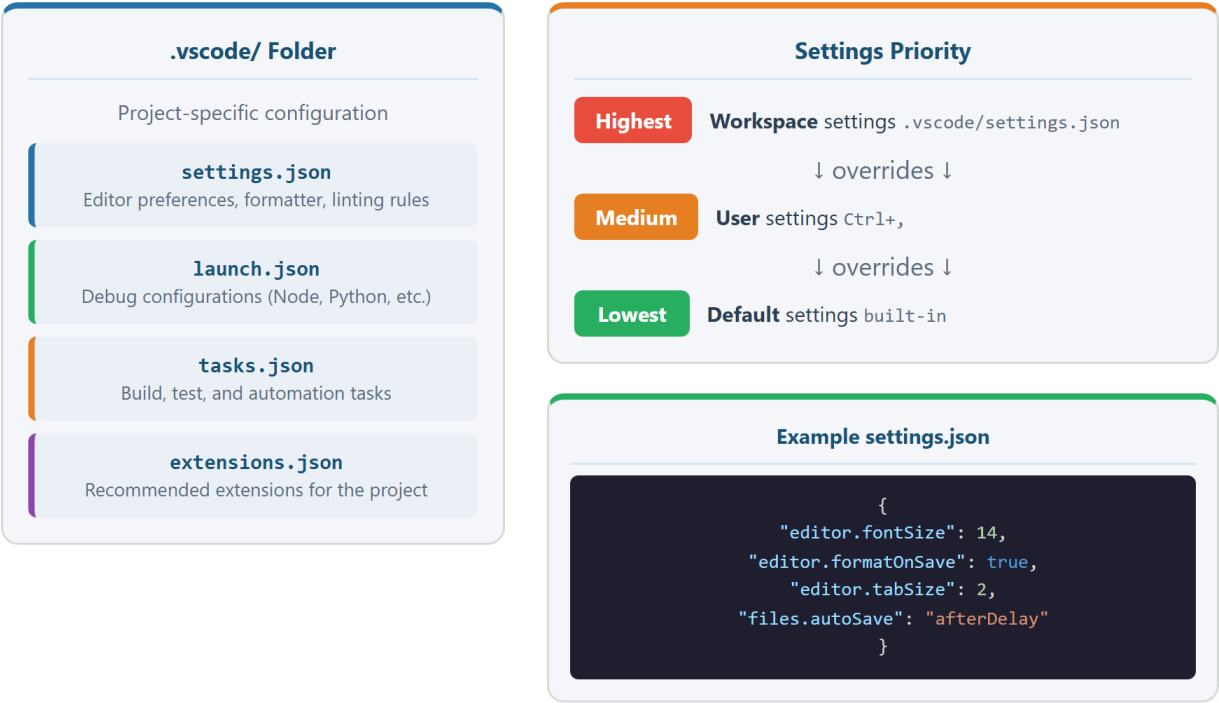
Debug & Git	
F5	Start debugging
F9	Toggle breakpoint
F10 / F11	Step over / into
Ctrl+Shift+G	Source Control
Ctrl+Shift+X	Extensions

Full reference: Ctrl+K Ctrl+S to open Keyboard Shortcuts editor



Workspace & Settings

Workspace & Settings



Productivity Tips

- Zen Mode — distraction-free editing (Ctrl+K Z)
- Emmet abbreviations — type `ul>li*3` and press Tab for HTML boilerplate
- Snippets — type `'log'` and Tab for `console.log()` (customizable)
- Settings Sync — sign in with GitHub/Microsoft to sync across machines
- Profiles — create separate settings profiles for different projects
- Remote Development — edit files on servers/containers via SSH



Class Exercises

Section 9



Exercise 1: Explore the VS Code UI

Goal

- Become comfortable navigating the VS Code interface

Steps

- Open VS Code and create a new folder called 'vscode-practice'
- Open the Command Palette (Ctrl+Shift+P) and change the color theme
- Open the integrated terminal (Ctrl+`) and run: `echo 'Hello from VS Code'`
- Create a new file `index.html` using the Explorer sidebar
- Toggle the Side Bar (Ctrl+B), Panel, and Activity Bar on/off
- Use Quick Open (Ctrl+P) to jump to your file by typing its name



Exercise 2: Editing Power Features

Goal

- Practice multi-cursor, find/replace, and navigation shortcuts

Steps

- Create a file with 10 lines of sample data (e.g., names or items)
- Use Ctrl+D to select and rename a repeated word across the file
- Use Alt+Up/Down to move lines around
- Use Shift+Alt+Down to duplicate a line
- Open Find & Replace (Ctrl+H) — replace all occurrences of a word
- Use Ctrl+Shift+K to delete lines quickly



Exercise 3: Extensions & Customization

Goal

- Install useful extensions and customize your environment

Steps

- Open the Extensions view (Ctrl+Shift+X)
- Search for and install: Prettier, Code Spell Checker, Material Icon Theme
- Create an HTML file, type ! and press Tab to generate boilerplate (Emmet)
- Open Settings (Ctrl+,) and enable 'Format On Save'
- Write some messy JavaScript — save and watch Prettier format it
- Explore the Settings UI — try changing font size and theme



Exercise 4: Debugging a Program

Goal

- Use the VS Code debugger to find and fix a bug

Steps

- Create a file called debug-practice.js with a simple function
- Add a loop that calculates a sum (intentionally include an off-by-one error)
- Click the gutter to set a breakpoint inside the loop
- Press F5 → select 'Node.js' to start debugging
- Step through each iteration with F10 — watch variables change
- Add a Watch expression to monitor the running total
- Fix the bug and re-run to verify



Key Takeaways

Key Takeaways

1. VS Code is a lightweight, powerful IDE used by most developers
2. The Command Palette (Ctrl+Shift+P) is your shortcut to everything
3. Multi-cursor editing and keyboard shortcuts boost productivity
4. Extensions customize VS Code for any language or workflow
5. The built-in debugger replaces print-statement debugging
6. Git integration lets you stage, commit, and push without leaving the editor

